

Proyecto

AquaSenseCloud

Infraestructura para la
Computación de Altas Prestaciones
Curso: 2025/2026

Javier Hernández Rosique
Álvaro Espejo Martínez

ÍNDICE

<i>Introducción al Proyecto</i>	2
<i>Plan de trabajo</i>	3
<i>Visualización de la arquitectura</i>	5
<i>Explicación de la arquitectura</i>	7
<i>Implementación</i>	12
Paso 1: Despliegue del stack proy-icap-red:	13
Paso 2: Despliegue del stack proy-icap-server:.....	13
Paso 3: Despliegue del stack proy-icap-pipeline:	14
Paso 4: Especificación del desencadenante de Lambda1:	15
Paso 5: Ingesta de datos:	15
Paso 6: Verificación:	16
<i>Resumen de recursos y servicios desplegados:</i>	18
<i>Anexo de replicación</i>	23
<i>Anexo de automatización</i>	25

Introducción al Proyecto

En este proyecto abordamos el diseño y la implementación de una infraestructura capaz de procesar datos ambientales de manera automática, eficiente y escalable. Partimos de un conjunto de datos generados por balizas situadas en el Mar Menor, los cuales se almacenan inicialmente en un fichero CSV. Nuestro objetivo es construir un sistema que permita **ingestar estos datos de forma automática, procesarlos siguiendo los requisitos definidos en el enunciado, detectar posibles anomalías y poner la información resultante a disposición de los usuarios mediante una API accesible y fiable**.

Para lograrlo, desarrollamos una arquitectura basada en servicios de AWS que integra distintos componentes especializados: almacenamiento en S3, procesamiento serverless mediante funciones Lambda, persistencia en DynamoDB, contenedores desplegados en un clúster ECS con balanceo de carga, y redes privadas con control exhaustivo del tráfico. Además, incorporamos mecanismos de **alerta, escalabilidad automática, monitorización en tiempo real y optimización de costes**, como el uso de VPC Gateway Endpoints para evitar tráfico innecesario a través del NAT Gateway.

Nuestro objetivo final es construir un **pipeline de datos completo**, capaz de funcionar de extremo a extremo: desde la subida de los datos crudos hasta su consulta final por parte del cliente. Todo el despliegue se realiza mediante **Infrastructure as Code (IaC)** utilizando plantillas de CloudFormation, de forma que la solución sea reproducible, trazable y fácil de mantener.

A lo largo de esta memoria describimos las decisiones de diseño que hemos adoptado, la arquitectura resultante, el proceso de implementación y los mecanismos de automatización y monitorización que garantizan el correcto funcionamiento del sistema.

Plan de trabajo

La siguiente tabla recoge la distribución de tareas realizadas por los miembros del equipo, junto con una breve descripción de cada una de ellas.

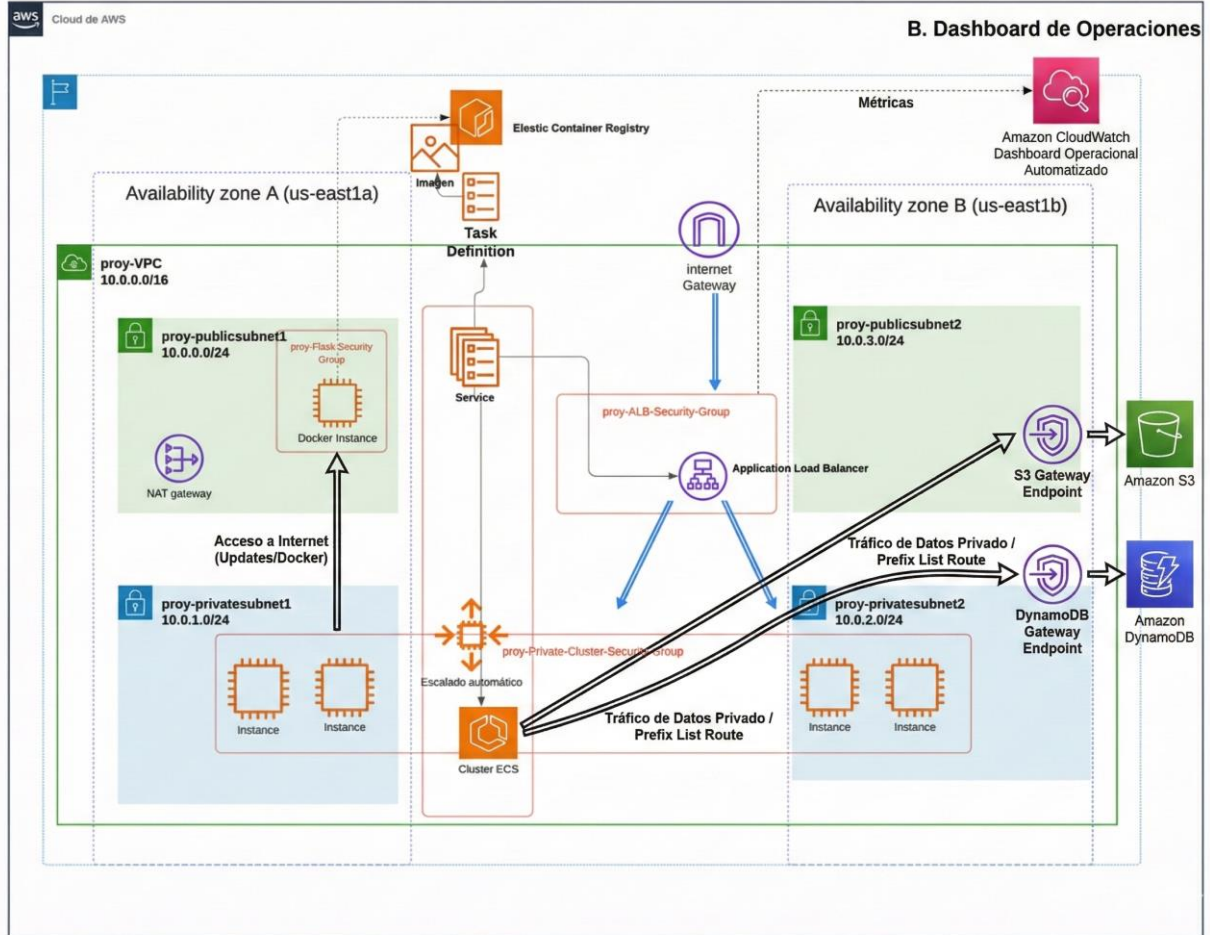
Tarea	Miembro responsable	Descripción de la tarea
Planificación inicial del proyecto	Conjunto	Definición del alcance, estructura del trabajo, reparto inicial de responsabilidades y análisis del enunciado.
Implementación de Lambda 1 y Lambda 2	Álvaro	Desarrollo de las funciones de ingesta, normalización y procesamiento de métricas mensuales sobre DynamoDB.
Desarrollo de la aplicación Flask y verificación del resultado final	Javier	Implementación de la API Flask, conexión con DynamoDB y validación de las rutas de consulta.
Implementación de Lambda 3 y automatización del pipeline	Conjunto	Desarrollo del sistema de detección de anomalías, integración con SNS y encadenamiento de eventos mediante Streams.
Elaboración de la presentación final	Conjunta	Preparación del material expositivo con explicación técnica del pipeline, arquitectura y resultados.
Automatización de la infraestructura	Álvaro	Creación de las plantillas IaC para VPC, ECS, endpoints, dashboard y pipeline completo.

Infraestructura para la Computación de Altas Prestaciones

mediante CloudFormation		
Revisión funcional y pruebas del sistema completo	Conjunto	Validación conjunta del comportamiento del pipeline, pruebas de consultas, alertas y funcionamiento del clúster ECS.
Redacción y estructuración de la memoria	Conjunto	Elaboración del documento final, integración de las secciones y revisión de calidad.

Visualización de la arquitectura

Arquitectura de la solución



Explicación de la arquitectura

Infraestructura:

Nuestra solución se basa en una infraestructura desplegada dentro de una VPC que actúa como entorno aislado y seguro para todos los recursos del proyecto. En ella definimos dos subredes públicas y dos subredes privadas, distribuidas en dos zonas de disponibilidad distintas con el fin de mejorar la tolerancia a fallos y garantizar la continuidad del servicio ante posibles incidencias en una de las zonas. Sobre estas subredes habilitamos un **Internet Gateway** para permitir el tráfico de entrada y salida en las subredes públicas, así como un **NAT Gateway** que proporciona acceso a Internet a las instancias situadas en las subredes privadas, necesario para que puedan registrarse correctamente en el clúster ECS.

Dado que exponer recursos privados al exterior puede suponer un riesgo desde el punto de vista de la seguridad, asignamos a las instancias del clúster un grupo de seguridad específico que únicamente permite tráfico entrante procedente del balanceador de carga, limitado al puerto 5000 utilizado por nuestro servidor Flask. De esta forma evitamos accesos no autorizados y reforzamos el aislamiento de los componentes internos.

Con el objetivo de optimizar tanto la seguridad como el coste de las operaciones de lectura y escritura en servicios gestionados de AWS, adoptamos una arquitectura de red híbrida. Mientras que el tráfico de mantenimiento que requieren las instancias (por ejemplo, descargas desde repositorios externos) se canaliza a través del NAT Gateway, todas las comunicaciones con S3 y DynamoDB se realizan mediante **VPC Gateway Endpoints**. Esto nos permite mantener el tráfico de datos dentro de la red interna de AWS, reduciendo la latencia, eliminando la dependencia del NAT para grandes volúmenes de transferencia y evitando costes adicionales de salida.

La aplicación Flask que expone las funcionalidades del proyecto se ejecuta en contenedores Docker alojados en un repositorio ECR. A partir de esta imagen definimos la tarea ECS que será desplegada en un clúster de instancias EC2 ubicado íntegramente en las subredes privadas. Para garantizar la disponibilidad del servicio y distribuir la carga de forma uniforme, situamos un balanceador de carga en las subredes públicas. Este componente no solo se encarga de dirigir las peticiones entrantes hacia las instancias del clúster, sino que además constituye el único punto de acceso permitido a la capa de aplicación, reforzando así el modelo de seguridad.

Inicialmente configuramos el clúster con cuatro instancias EC2, aunque habilitamos la posibilidad de escalar hasta seis en función de la demanda. Este rango se ha definido pensando en un entorno destinado a pruebas y validación, más que a un despliegue productivo. Por ello, hemos optado por una infraestructura ajustada, centrada en cubrir los requisitos funcionales de la práctica manteniendo el coste operativo lo más reducido posible.

Pipeline:

Infraestructura para la Computación de Altas Prestaciones

El punto de partida de nuestro sistema es la llegada de los datos crudos. Cada vez que incorporamos un fichero CSV al bucket S3 configurado para este fin, se activa automáticamente el flujo de procesamiento que hemos diseñado. Esta acción desencadena el inicio del pipeline y permite que los datos pasen por las distintas fases de transformación y almacenamiento.

En lugar de utilizar una base de datos relacional, optamos por **Amazon DynamoDB** debido a varias ventajas clave para un escenario como el nuestro. Su modelo NoSQL, sin esquema rígido, ofrece la flexibilidad necesaria para adaptarse a posibles cambios en el formato de los datos a lo largo del tiempo. Además, su integración con **Streams** nos permite reaccionar inmediatamente ante cualquier inserción o actualización, habilitando un procesamiento incremental completamente automatizado. Esto, unido a la eficiencia de las consultas basadas en clave, facilita obtener de manera rápida todos los registros asociados a un mes concreto, lo cual es fundamental para el cálculo de las métricas solicitadas.

Con el objetivo de reforzar la fiabilidad operativa y mejorar la capacidad de diagnóstico del sistema, incorporamos un módulo de monitorización centralizada, creamos un Dashboard. Para ello desplegamos, mediante Infrastructure as Code, un panel personalizado en Amazon CloudWatch que recoge indicadores esenciales del comportamiento de la infraestructura: número de invocaciones de cada fase del pipeline, tiempos de ejecución y utilización de CPU y memoria por parte del clúster ECS. Además, añadimos métricas específicas del servicio ECS, como el número de tareas activas y su estado, con el fin de verificar que el mecanismo de ajuste automático de capacidad funciona correctamente y que el balanceador distribuye la carga entre instancias de forma uniforme. Esta capa de observabilidad nos permite identificar anomalías, anticipar saturaciones y validar que la infraestructura responde adecuadamente a la carga de trabajo en todo momento.

Toda la lógica del pipeline descansa sobre **tres funciones Lambda**, cada una responsable de una parte específica del procesamiento y coordinadas entre sí mediante eventos del propio sistema. Estas funciones actúan como etapas encadenadas que permiten que los datos avancen progresivamente desde su estado original en el CSV hasta el almacenamiento final de los resultados agregados que consulta la aplicación web.

1. Lambda1:

La primera función Lambda actúa como punto de entrada del pipeline. Su ejecución se desencadena automáticamente cuando incorporamos un nuevo fichero CSV en nuestro bucket S3 de ingesta (*proydatabucket-javier-alvaro-v2*). Este comportamiento se debe a la integración nativa entre S3 y Lambda, que permite ejecutar código de forma inmediata sin necesidad de servidores adicionales.

El propósito principal de esta función es trasladar los datos crudos a la tabla DynamoDB *proy-Datos*, que constituye la base del procesamiento posterior. Para ello, la función lee el contenido del CSV y organiza los registros en lotes antes de insertarlos mediante un **batch writer**. Optamos por este mecanismo por razones tanto de rendimiento como de consistencia:

- La escritura en lotes reduce el número de operaciones I/O y acelera la inserción.

Infraestructura para la Computación de Altas Prestaciones

- Evita que DynamoDB Streams pierda eventos cuando se generan muchas inserciones consecutivas, algo que podría ocurrir si se escribiera registro a registro.

Este detalle es especialmente relevante porque las siguientes etapas del pipeline dependen de que el stream reciba **todas** las modificaciones en el orden adecuado.

Además, durante esta fase realizamos un proceso de normalización del campo fecha, originalmente en el formato *aaaa-mm-dd*. Con el fin de optimizar las consultas mensuales que se ejecutarán más adelante, dividimos este valor en dos atributos:

- YearMonth, que utilizamos como clave de partición,
- Day, que usamos como clave de ordenación.

Esta decisión de modelado responde a buenas prácticas de diseño en DynamoDB, donde una única clave de partición que agrupa todos los registros de un mes permite resolver las consultas con una sola operación eficiente (*Query*) en lugar de escaneos más costosos.

En resumen, Lambda 1 garantiza la correcta ingesta, transformación y persistencia de los datos de origen, preparando la información para las fases de análisis del pipeline.

2. Lambda2:

La segunda función Lambda constituye el motor de procesamiento del sistema. Su activación se produce cada vez que la tabla *proy-Datos* experimenta un cambio, ya sea la inserción de un nuevo registro o la modificación de uno existente. Esto es posible gracias a DynamoDB Streams, que genera un evento por cada operación realizada sobre la tabla y lo envía a Lambda para su procesamiento.

Cada ejecución de esta función recibe un lote de eventos y, para cada uno de ellos, obtiene el valor modificado junto con su clave YearMonth. A partir de esta clave, la función recupera todos los registros pertenecientes al mes correspondiente mediante una operación *Query*, así como datos del mes anterior (necesarios para calcular la diferencia de máximas temperaturas) y, en casos concretos, del mes siguiente (si la modificación afecta al máximo mensual y puede alterar el cálculo futuro).

Con esta información, Lambda 2 calcula de forma incremental las métricas exigidas por la práctica:

- temperatura media del mes,
- desviación estándar del conjunto de muestras,

Infraestructura para la Computación de Altas Prestaciones

- diferencia entre la temperatura máxima del mes y la del mes anterior.

Una vez obtenidos los valores actualizados, la función los almacena en la tabla *proy-Resultados*. Esta tabla representa la capa de datos ya procesados y es la que consulta el servicio Flask desplegado en ECS cuando recibe peticiones de los usuarios.

El uso de DynamoDB Streams nos permite adoptar un enfoque **reactivo e incremental**, en el que solo procesamos los elementos modificados en lugar de recalculamos todo el histórico. Esto reduce de forma significativa el consumo de recursos y dota al sistema de una mayor eficiencia.

3. Lambda3:

La tercera función Lambda tiene un propósito distinto al de las anteriores: actúa como un sistema de alerta temprana. También se activa directamente desde DynamoDB Streams cuando *proy-Datos* incorpora nuevos registros o se actualizan los existentes. Su misión es detectar posibles anomalías en las desviaciones semanales registradas.

Durante cada invocación, la función analiza el lote de eventos recibido —con un máximo configurado de 100 elementos por ejecución— y verifica si algún registro presenta una desviación superior al umbral de 0.5. De cumplirse esta condición, la función reúne todas las fechas afectadas y construye un mensaje de aviso que envía a Amazon SNS. Este servicio se encarga de distribuir la alerta por correo electrónico a los suscriptores correspondientes.

La decisión de procesar los eventos en lotes responde a la necesidad de limitar el número de correos generados. Dado que el sistema podría recibir múltiples registros consecutivos, agruparlos en un mismo mensaje evita saturar a los usuarios con alertas repetitivas y reduce la carga sobre el servicio de notificaciones.

En conjunto, Lambda 3 aporta una capa adicional de vigilancia que permite detectar situaciones anómalas en el entorno de estudio, cumplir con los requisitos del enunciado y completar el ciclo de monitorización del pipeline.

Flask:

La capa de servicio de nuestra solución está implementada mediante una aplicación Flask que se despliega como contenedor Docker dentro del clúster ECS. Esta aplicación actúa como interfaz de consulta para los analistas de datos, proporcionando acceso a la información ya procesada y almacenada en la tabla *proy-Resultados* de DynamoDB.

El servicio se integra directamente con DynamoDB utilizando como clave de partición el campo YearMonth, lo que permite recuperar los datos mensuales mediante consultas muy eficientes (*Query*) sin necesidad de recorrer toda la tabla. De esta manera, la API expone únicamente datos finales ya calculados por el pipeline, evitando operaciones de procesamiento en tiempo real y reduciendo la carga de cómputo en las instancias EC2 del clúster.

Infraestructura para la Computación de Altas Prestaciones

Para asegurar la disponibilidad del servicio y distribuir correctamente las peticiones, la aplicación Flask se ejecuta detrás de un Application Load Balancer (ALB). El ALB enruta el tráfico entrante hacia las instancias del clúster en las subredes privadas, garantizando un único punto de acceso y contribuyendo al diseño de seguridad de la arquitectura. Además, cada respuesta incluye la dirección IP privada de la instancia EC2 que atendió la petición, lo que facilita el diagnóstico y la monitorización del comportamiento del clúster.

La API implementa las siguientes rutas:

1. Ruta raíz (/)

Sirve como punto de entrada del servicio. Si durante la ejecución de cualquier consulta se produce un error — como solicitar datos inexistentes o parámetros incorrectos —, el cliente es redirigido nuevamente a esta ruta. De esta forma se evita devolver respuestas incompletas y se mantiene un comportamiento consistente ante fallos.

2. Ruta /maxdiff

Permite consultar la diferencia entre la temperatura máxima del mes solicitado y la del mes anterior. El cliente debe proporcionar los parámetros **year** y **month** en la cadena de consulta. La función recupera de *proy-Resultados* el valor precalculado por el pipeline y lo devuelve en formato JSON.

3. Ruta /sd

Devuelve la desviación estándar de las muestras del mes indicado. Al igual que en el resto de rutas, la consulta se realiza directamente sobre los valores procesados almacenados en DynamoDB, sin necesidad de recálculo.

4. Ruta /temp

Proporciona la temperatura media mensual. La función consulta el registro correspondiente y responde con la media precalculada.

Todas las rutas siguen el mismo patrón de respuesta:

- devuelven un objeto JSON,
- incluyen el valor solicitado (maxdiff, sd o temp),
- incorporan la fecha consultada,
- e indican la IP privada de la instancia EC2 que procesó la petición.

Este último aspecto es especialmente útil para comprobar el correcto funcionamiento del balanceo de carga y verificar que el clúster distribuye adecuadamente las solicitudes entre las distintas instancias.

Infraestructura para la Computación de Altas Prestaciones

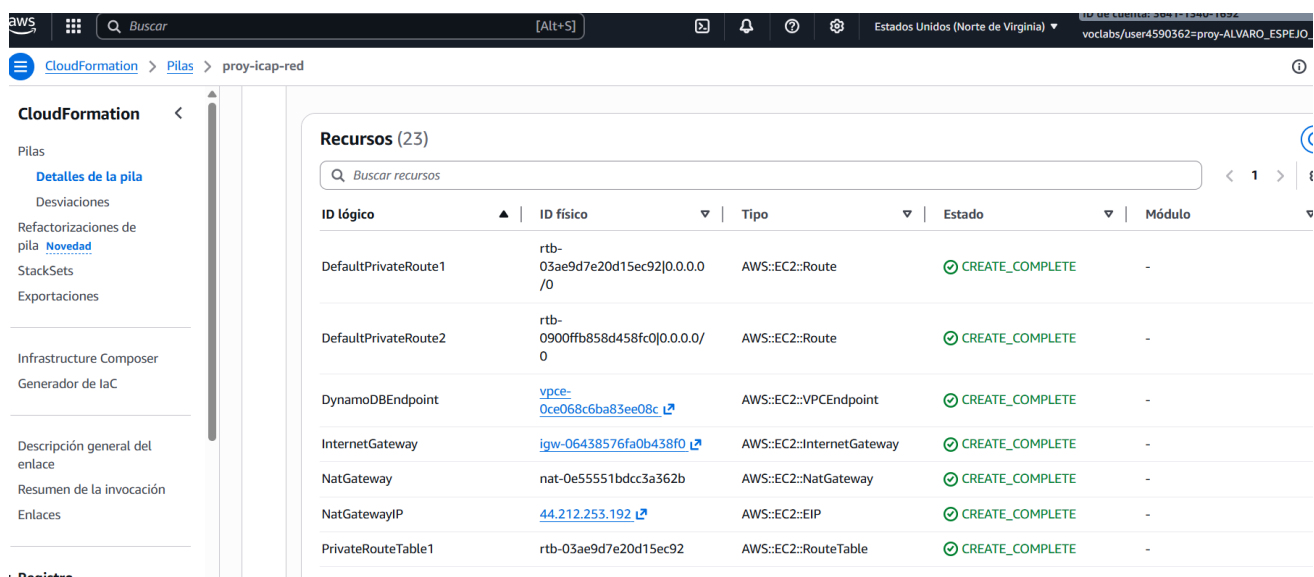
Implementación

Paso 1: Despliegue del stack proy-icap-red:

Esta plantilla despliega la arquitectura de red base (VPC, subredes públicas y privadas, Gateways).

Configura automáticamente las tablas de enrutamiento privadas inyectando rutas hacia los VPC Endpoints mediante Prefix Lists

NOTA: El rol de IAM en todas las plantillas es *LabRole*.



ID lógico	ID físico	Tipo	Estado	Módulo
DefaultPrivateRoute1	rtb-03ae9d7e20d15ec92 0.0.0.0/0	AWS::EC2::Route	CREATE_COMPLETE	-
DefaultPrivateRoute2	rtb-0900ffb858d458fc0 0.0.0.0/0	AWS::EC2::Route	CREATE_COMPLETE	-
DynamoDBEndpoint	vpce-0ce068c6ba83ee08c	AWS::EC2::VPCEndpoint	CREATE_COMPLETE	-
InternetGateway	igw-06438576fa0b438f0	AWS::EC2::InternetGateway	CREATE_COMPLETE	-
NatGateway	nat-0e55551bdcc3a362b	AWS::EC2::NatGateway	CREATE_COMPLETE	-
NatGatewayIP	44.212.253.192	AWS::EC2::EIP	CREATE_COMPLETE	-
PrivateRouteTable1	rtb-03ae9d7e20d15ec92	AWS::EC2::RouteTable	CREATE_COMPLETE	-

rtb-03ae9d7e20d15ec92 / proy-privateroutetable1

Detalles Información

ID de tabla de enrutamiento

[rtb-03ae9d7e20d15ec92](#)

Principal

☐ No

VPC

[vpc-04b3965d6cb1725fc](#) | proy-VPC

ID de propietario

[364113401692](#)

Rutas

Asociaciones de subredes

Asociaciones de borde

Propagación

Rutas (4)

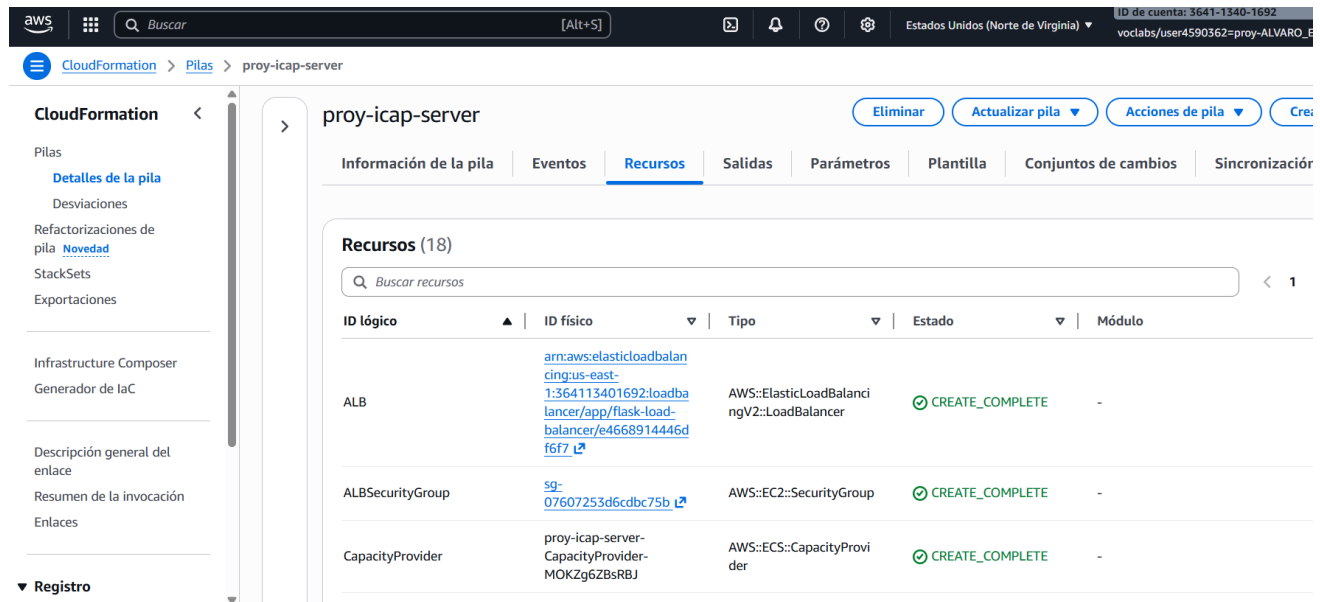
Podemos observar que se incluyen rutas hacia [vpce-0...](#), qué son los Endpoints para el tráfico de servicios de AWS, desviándolo del NAT Gateway para mayor optimización.

Destino	Objetivo	Estado
pl-02cd2c6b	vpce-0ce068c6ba83ee08c	Activo
pl-63a5400a	vpce-091d8668dd802afb2	Activo
0.0.0.0/0	nat-0e55551bdcc3a362b	Activo
10.0.0.0/16	local	Activo

Infraestructura para la Computación de Altas Prestaciones

Paso 2: Despliegue del stack proy-icap-server:

Esta plantilla aprovisiona la capa de computación: el servidor Docker (Bastion), el clúster ECS, el Auto Scaling Group y el Balanceador de Carga (ALB). Además, se ha incorporado en la definición de la infraestructura (**laC**) el despliegue automático de un Dashboard de Operaciones en Amazon CloudWatch.



ID lógico	ID físico	Tipo	Estado	Módulo
ALB	arn:aws:elasticloadbalancing:us-east-1:364113401692:loadbalancer/app/flask-loadbalancer/e466891444df6f7	AWS::ElasticLoadBalancingV2::LoadBalancer	CREATE_COMPLETE	-
ALBSecurityGroup	sg-07607253d6cdbc75b	AWS::EC2::SecurityGroup	CREATE_COMPLETE	-
CapacityProvider	proy-icap-server-CapacityProvider-MOKZg6ZBsRBJ	AWS::ECS::CapacityProvider	CREATE_COMPLETE	-

Paso 3: Despliegue del stack proy-icap-pipeline:

Con esta plantilla vamos a crear todo lo que necesitamos: desde las tablas DynamoDB para guardar la información (cruda y procesada) y el bucket de S3, hasta las funciones Lambda con su código listo. Lo más importante es que deja todo conectado para que, al llegar datos nuevos a la tabla principal, las funciones se activen solas.

Nota: Una vez desplegado, será necesario aceptar la suscripción al servicio SNS mediante el email que se recibe al correo electrónico correspondiente.

Infraestructura para la Computación de Altas Prestaciones

CloudFormation > Pilas > proy-icap-pipeline

Eliminar Actualizar pila Acciones de pila

Información de la pila Eventos Recursos Salidas Parámetros Plantilla Conjuntos de cambios Si

Recursos (10)

Buscar recursos

ID lógico	ID físico	Tipo	Estado	Módulo
DynamoDBTable1	proy-Datos	AWS::DynamoDB::Table	CREATE_COMPLETE	-
DynamoDBTable2	proy-Resultados	AWS::DynamoDB::Table	CREATE_COMPLETE	-
EmailSubscription	arn:aws:sns:us-east-1:364113401692:DeviationExceeded:114155c1-7926-443d-9634-c62ce082787c	AWS::SNS::Subscription	CREATE_COMPLETE	-
Lambda1Function	lambda1	AWS::Lambda::Function	CREATE_COMPLETE	-
Lambda2Function	lambda2	AWS::Lambda::Function	CREATE_COMPLETE	-

Paso 4: Especificación del desencadenante de Lambda1:

Es necesario especificar manualmente que el trigger de la LambdaFunction es la ingesta de datos al bucket que, previamente, crea la plantilla de CloudFormation.

Lambda > Funciones > lambda1

Esta función pertenece a una aplicación. [Haga clic aquí](#) para administrarla.

▼ Información general de la función Información

Diagrama Plantilla

lambda1

Layers (0)

S3

+ Agregar desencadenador

Funciones relacionadas: Seleccionar una función

+ Agregar destino

Paso 5: Ingesta de datos:

Una vez completado lo descrito, el proyecto está desplegado. Verificamos el correcto funcionamiento inyectando datos en el bucket para verificar el desencadenante de la Lambda y todo lo posterior.

Infraestructura para la Computación de Altas Prestaciones

proydatabucket-javier-alvaro-v2 [Información](#)

[Objetos](#) | [Metadatos](#) | [Propiedades](#) | [Permisos](#) | [Métricas](#) | [Administración](#) | [Puntos de acceso](#)

Objetos (1)

[Copiar URI de S3](#) [Copiar URL](#) [Descargar](#) [Abrir](#) [Eliminar](#) [Acciones](#) [Crear carpeta](#) [Cargar](#)

Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Puede utilizar el [inventario de Amazon S3](#) para obtener una lista de todos los objetos de su bucket. Para que otras personas obtengan acceso a sus objetos, tendrá que concederles permisos de forma explícita. [Más información](#)

Buscar objetos por prefijo

☐	Nombre	Tipo	Última modificación	Tamaño	Clase de almacenamiento
☐	Temperatura.csv	csv	10 Dec 2025 5:49:42 PM CET	18.1 KB	Estándar

Paso 6: Verificación:

Para comprobar que todo ha funcionado como debe, verificaremos lo siguiente:

- Se han insertado datos en la tabla de DynamoDB.
- Hemos recibido una notificación por correo con las desviaciones mayores a 0'5.
- Validación de la API.
- Dashboard de CloudWatch con datos.

Tabla: proy-Resultados: elementos devueltos (50)

El análisis inició el diciembre 11, 2025, 19:47:46

☐	YearMonth (Cadena)	maxdiff	sd	temp
☐	2022-06	3.1176218...	0.4882785...	26.90669854192887
☐	2019-01	-2.0572795...	0.5686461...	11.81950330734253
☐	2018-06	3.7531986...	0.6554430...	25.67467975616455
☐	2019-09	-0.6271845...	0.7189719...	25.555569856620448
☐	2024-07	2.7110900...	0.3635705...	28.33078333556377
☐	2020-08	1.0229026...	0.3099586...	29.574033567394604
☐	2018-02	0.3896799...	0.4211053...	12.22459856669108
☐	2018-05	5.2885761...	0.3986176...	21.659515380859375

Infraestructura para la Computación de Altas Prestaciones



AWS Notifications <no-reply@sns.amazonaws.com>

para mí ▾



Deviation exceeded on the following dates:

2024-01-23: 0.594

2024-04-29: 0.732

2024-05-22: 0.570

2024-06-25: 0.542

--

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:

<https://sns.us-east-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:us-east-1:36411340>

Dar formato al texto ☐

```
{
  "Date": "2023-01",
  "Maxdiff": "-0.9648021288236155",
  "ip": "10.0.2.120"
}
```

NOTA: La consulta debe realizarse según la estructura

<http://54.91.63.31:5000/maxdiff?month=1&year=2023>, siendo 54.91.63.31:5000 la IPv4 del balanceador de carga.

(las consultas también admiten por temperatura y por sd).

Infraestructura para la Computación de Altas Prestaciones

Actividad del Pipeline de Datos (Invocaciones)

5 minutos ▼

Suma ▼

1h

3h

12h

1d

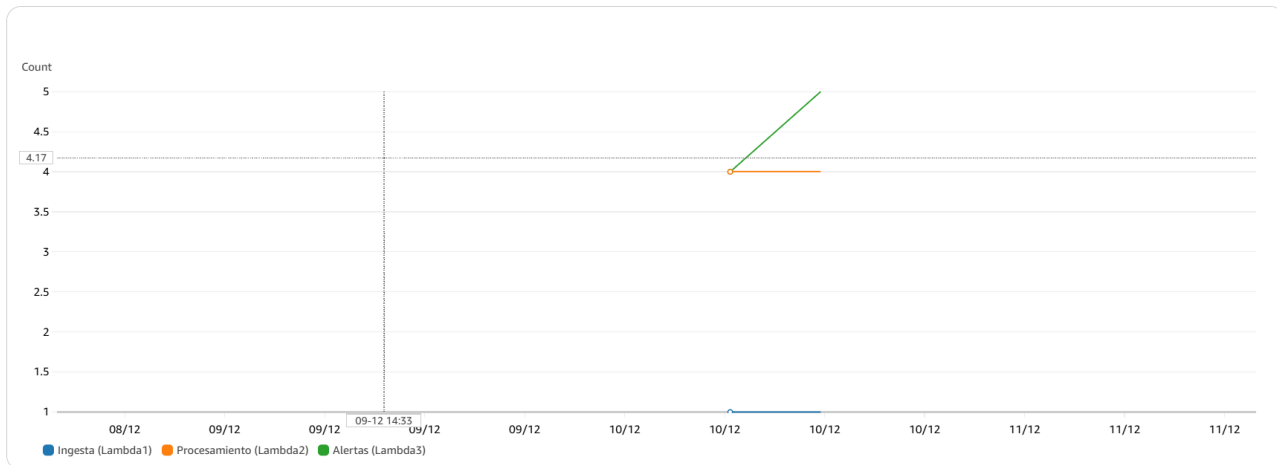
3d

1sem.

Personalizado

Zona horaria UTC ▼

✕

[Ver en métricas](#) [Cerrar](#)

Salud del Clúster ECS

5 minutos ▼

Media ▼

1h

3h

12h

1d

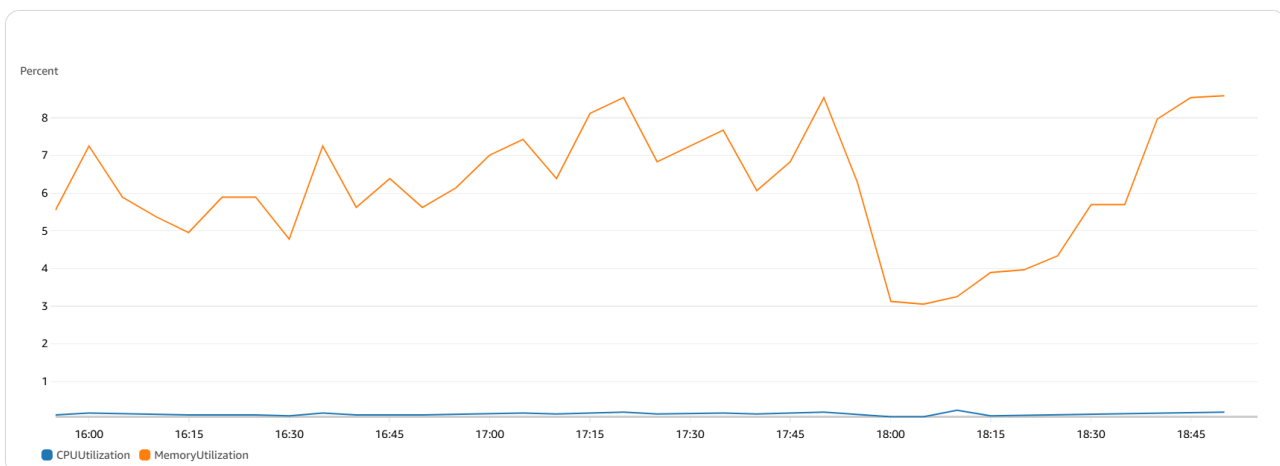
3d

1sem.

Personalizado

Zona horaria UTC ▼

✕

[Ver en métricas](#) [Cerrar](#)

En la primera gráfica del Dashboard, miramos las tres observaciones de activaciones de las funciones Lambda (Lambda1, Lambda2, Lambda3).

En la segunda gráfica, tenemos información útil sobre el consumo de memoria y de CPU.

Resumen de recursos y servicios desplegados:

Identificador de recurso	Nombre de recurso	Tipo de recurso en AWS	Comentario (opcional)
R01	proy-Datos	DynamoDB::Table	Tabla donde almacenamos los datos crudos ingeridos desde los ficheros CSV.
R02	proy-Resultados	DynamoDB::Table	Tabla donde almacenamos los datos ya procesados y listos para ser consultados por la API Flask.
R03	proydatabucket-javier-alvaro-v2	S3::Bucket	Bucket de entrada donde el equipo sube los ficheros CSV con los datos semanales del Mar Menor.
R04	SNSTopic	SNS::Topic	Tópico SNS usado para publicar las alertas cuando la desviación semanal supera el umbral.
R05	EmailSubscription	SNS::Subscription	Suscripción por correo electrónico al tópico SNSTopic para que el profesor reciba las alarmas.

Infraestructura para la Computación de Altas Prestaciones

R06	lambda1	Lambda::Function	Función que ingiere el CSV desde S3 y carga los datos normalizados en la tabla proy-Datos .
R07	lambda2	Lambda::Function	Función que calcula las métricas mensuales y actualiza la tabla proy-Resultados .
R08	lambda3	Lambda::Function	Función que detecta desviaciones $> 0,5$ y dispara el envío de notificaciones a través de SNS.
R09	proy-VPC	EC2::VPC	VPC que aísla lógicamente toda la infraestructura de red del proyecto.
R10	proy-publicsubnet1	EC2::Subnet	Subred pública de la VPC en la zona de disponibilidad A, usada entre otros por el ALB.
R11	proy-publicsubnet2	EC2::Subnet	Subred pública de la VPC en la zona de disponibilidad B para alta disponibilidad del ALB.

Infraestructura para la Computación de Altas Prestaciones

R12	proy-privatesubnet1	EC2::Subnet	Subred privada para una de las zonas del clúster ECS.
R13	proy-privatesubnet2	EC2::Subnet	Subred privada para la segunda zona del clúster ECS.
R14	S3Endpoint	EC2::VPCEndpoint	Endpoint de tipo Gateway para acceso privado y optimizado a S3 desde las subredes privadas.
R15	DynamoDBEndpoint	EC2::VPCEndpoint	Endpoint de tipo Gateway para acceso privado a DynamoDB sin pasar por la NAT Gateway.
R16	proy-repo	ECR::Repository	Repositorio ECR donde se almacena la imagen Docker de la aplicación Flask.
R17	proy-DockerServer	EC2::Instance	Instancia que construye la imagen Docker y la envía al repositorio proy-repo.
R18	proy-Flask Security Group	EC2::SecurityGroup	Grupo de seguridad de la instancia proy-DockerServer para acceso de administración.

Infraestructura para la Computación de Altas Prestaciones

R19	proy-ALB-Security-Group	EC2::SecurityGroup	Grupo de seguridad del Application Load Balancer, que expone el puerto 5000 al exterior.
R20	proy-Private-Cluster-Security-Group	EC2::SecurityGroup	Grupo de seguridad de las instancias del clúster ECS; solo acepta tráfico desde el ALB.
R21	proy-ecs-cluster	ECS::Cluster	Clúster ECS que agrupa las instancias EC2 que ejecutan las tareas de la aplicación Flask.
R22	proy-task	ECS::TaskDefinition	Definición de la tarea ECS: imagen, puertos, CPU/memoria y variables de entorno del contenedor.
R23	flask-load-balancer	ElasticLoadBalancingV2::LoadBalancer	Application Load Balancer que distribuye las peticiones HTTP entre las tareas del clúster ECS.
R24	ip-target-group	ElasticLoadBalancingV2::TargetGroup	Grupo de destino del ALB, configurado para registrar las tareas ECS por IP en el puerto 5000.

Infraestructura para la Computación de Altas Prestaciones

R25	proy-ecs-service	ECS::Service	Servicio ECS que mantiene el número deseado de tareas y las integra con el ALB.
R26	ProjectDashboard	CloudWatch::Dashboard	Panel operativo que muestra invocaciones del pipeline y métricas de CPU/memoria del clúster ECS.

Anexo de replicación

- 1.- Desplegar la plantilla *proy-icap-red*. Rol IAM LabRole
- 2.- Desplegar la plantilla *proy-icap-server*. Rol IAM LabRole. Como parámetro **NetworkStackName**, importante que sea el mismo que *proy-icap-red*.
- 3.- Desplegar la plantilla *proy-icap-pipeline*. Rol IAM LabRole
- 4.- Añadir desencadenante de Lambda1 (bucket de S3).
- 5.- Aceptar suscripción email SNS.
- 6.- Subir fichero csv al bucket S3.

NOTA: A la hora de la replicación, en nuestro caso, hemos tenido el problema de que los nombres de los buckets en S3 deben ser únicos a nivel global, y si está desplegado en una cuenta, da error en otra, ya que el nombre viene definido en las plantillas de CloudFormation (*hard coding*).

(hemos intentado añadirlo como parámetro para no tener que editar la plantilla pero no hemos podido).

Para asegurar el no afrontar este error en el despliegue, se deberá editar en la propia plantilla lo siguiente:

```
# --- Storage Layer (S3 & NoSQL) ---
S3Bucket:
  Type: AWS::S3::Bucket
  Properties:
    BucketName: proydatabucket-javier-alvaro-v2
```

Plantilla: Proy-icap-pipeline

```
# --- Compute Layer (Lambdas) ---
Lambda1Function:
  Type: AWS::Lambda::Function
  Properties:
    FunctionName: lambda1
    Handler: index.lambda_handler
    Role: !Sub arn:aws:iam::${AWS::AccountId}:role/LabRole
    Runtime: python3.12
    Timeout: 20
    Code:
```


Infraestructura para la Computación de Altas Prestaciones

```
ZipFile: |
import json, urllib, boto3, csv
import datetime
from decimal import Decimal
TABLE_NAME = 'proy-Datos'
BUCKET_NAME = 'proydatabucket-javier-alvaro-v2'
```

Plantilla: Proy-icap-pipeline

Anexo de automatización

Proy-icap-red

```
AWSTemplateFormatVersion: 2010-09-09
Description: >-
  Network Template: Base infrastructure definition (VPC, Subnets, Gateways and
  Routing).

# -----
# Outputs section
# -----

Outputs:
  VPC:
    Description: VPC ID
    Value: !Ref VPC
    Export:
      Name: !Sub '${AWS::StackName}-VPCID'

  PublicSubnet1:
    Description: Subnet ID for ALB and Bastion Host
    Value: !Ref PublicSubnet1
    Export:
      Name: !Sub '${AWS::StackName}-PublicSubnet1ID'

  PublicSubnet2:
    Description: Subnet ID for ALB High Availability
    Value: !Ref PublicSubnet2
    Export:
      Name: !Sub '${AWS::StackName}-PublicSubnet2ID'

  PrivateSubnet1:
    Description: Private Subnet ID for Cluster Zone A
    Value: !Ref PrivateSubnet1
    Export:
      Name: !Sub '${AWS::StackName}-PrivateSubnet1ID'

  PrivateSubnet2:
    Description: Private Subnet ID for Cluster Zone B
    Value: !Ref PrivateSubnet2
    Export:
      Name: !Sub '${AWS::StackName}-PrivateSubnet2ID'
```

Infraestructura para la Computación de Altas Prestaciones

```
# -----  
# Resources section  
# -----  
Resources:  
  
# --- Core Networking ---  
VPC:  
  Type: AWS::EC2::VPC  
  Properties:  
    CidrBlock: 10.0.0.0/16  
    EnableDnsSupport: true  
    EnableDnsHostnames: true  
    Tags:  
      - Key: Name  
        Value: proy-VPC  
  
# --- Gateways & Endpoints ---  
InternetGateway:  
  Type: AWS::EC2::InternetGateway  
  
VPCGatewayAttachment:  
  Type: AWS::EC2::VPCGatewayAttachment  
  Properties:  
    VpcId: !Ref VPC  
    InternetGatewayId: !Ref InternetGateway  
  
NatGatewayIP:  
  Type: AWS::EC2::EIP  
  DependsOn: VPCGatewayAttachment  
  Properties:  
    Domain: vpc  
  
NatGateway:  
  Type: AWS::EC2::NatGateway  
  Properties:  
    AllocationId: !GetAtt NatGatewayIP.AllocationId  
    SubnetId: !Ref PublicSubnet1  
  
# VPC Endpoints for S3 and DynamoDB (Cost & Security optimization)  
S3Endpoint:  
  Type: AWS::EC2::VPCEndpoint  
  Properties:  
    VpcId: !Ref VPC  
    ServiceName: !Sub com.amazonaws.${AWS::Region}.s3
```

Infraestructura para la Computación de Altas Prestaciones

```
VpcEndpointType: Gateway
RouteTableIds:
  - !Ref PrivateRouteTable1
  - !Ref PrivateRouteTable2

DynamoDBEndpoint:
  Type: AWS::EC2::VPCEndpoint
  Properties:
    VpcId: !Ref VPC
    ServiceName: !Sub com.amazonaws.${AWS::Region}.dynamodb
    VpcEndpointType: Gateway
    RouteTableIds:
      - !Ref PrivateRouteTable1
      - !Ref PrivateRouteTable2

# --- Subnet Definitions ---
PublicSubnet1:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: 10.0.0.0/24
    AvailabilityZone: !Select [ 0, !GetAZs { Ref: AWS::Region } ]
    Tags:
      - Key: Name
        Value: proy-publicsubnet1

PublicSubnet2:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: 10.0.3.0/24
    AvailabilityZone: !Select [ 1, !GetAZs { Ref: AWS::Region } ]
    Tags:
      - Key: Name
        Value: proy-publicsubnet2

PrivateSubnet1:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: 10.0.1.0/24
    AvailabilityZone: !Select [ 0, !GetAZs { Ref: AWS::Region } ]
    Tags:
      - Key: Name
```

Infraestructura para la Computación de Altas Prestaciones

```
Value: proy-privatesubnet1

PrivateSubnet2:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: 10.0.2.0/24
    AvailabilityZone: !Select [ 1, !GetAZs { Ref: AWS::Region } ]
    Tags:
      - Key: Name
        Value: proy-privatesubnet2

# --- Routing Configuration ---
PublicRouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VPC

PublicRoute:
  Type: AWS::EC2::Route
  DependsOn: VPCGatewayAttachment
  Properties:
    RouteTableId: !Ref PublicRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref InternetGateway

PrivateRouteTable1:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VPC
    Tags:
      - Key: Name
        Value: proy-privateroutetable1

PrivateRouteTable2:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VPC
    Tags:
      - Key: Name
        Value: proy-private-route-table2

DefaultPrivateRoute1:
  Type: AWS::EC2::Route
```

Infraestructura para la Computación de Altas Prestaciones

```
Properties:
  RouteTableId: !Ref PrivateRouteTable1
  DestinationCidrBlock: 0.0.0.0/0
  NatGatewayId: !Ref NatGateway

DefaultPrivateRoute2:
  Type: AWS::EC2::Route
  Properties:
    RouteTableId: !Ref PrivateRouteTable2
    DestinationCidrBlock: 0.0.0.0/0
    NatGatewayId: !Ref NatGateway

# --- Associations ---
PublicSubnetRouteTableAssociation1:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet1
    RouteTableId: !Ref PublicRouteTable

PublicSubnetRouteTableAssociation2:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet2
    RouteTableId: !Ref PublicRouteTable

PrivateSubnet1RouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    RouteTableId: !Ref PrivateRouteTable1
    SubnetId: !Ref PrivateSubnet1

PrivateSubnet2RouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    RouteTableId: !Ref PrivateRouteTable2
    SubnetId: !Ref PrivateSubnet2

PublicSubnetNetworkAclAssociation1:
  Type: AWS::EC2::SubnetNetworkAclAssociation
  Properties:
    SubnetId: !Ref PublicSubnet1
    NetworkAclId: !GetAtt [ VPC, DefaultNetworkAcl ]

PublicSubnetNetworkAclAssociation2:
```

Infraestructura para la Computación de Altas Prestaciones

```
Type: AWS::EC2::SubnetNetworkAclAssociation
Properties:
  SubnetId: !Ref PublicSubnet2
  NetworkAclId: !GetAtt [ VPC, DefaultNetworkAcl ]
```

Proy-icap-server

```
AWSTemplateFormatVersion: 2010-09-09
Description: >-
  Server Template: Provisions a scalable container infrastructure on AWS ECS.

# Resources deployed by this stack:
# --- Storage & Build ---
#   1 ECR Repository: To store the application Docker images.
#   1 EC2 Instance (Builder): Automates the Docker build and push process via
#   UserData.
# --- Compute & Orchestration ---
#   1 ECS Cluster: The logical grouping for the container tasks.
#   1 AutoScaling Group + Launch Template: Manages the lifecycle of the EC2
#   instances backing the cluster.
#   1 ECS Capacity Provider + Association: Links the ASG to the ECS Cluster for
#   infrastructure scaling.
#   1 ECS Task Definition: Defines the container specs (Image, CPU, Memory).
#   1 ECS Service: Orchestrates the task placement and integration with the Load
#   Balancer.
# --- Networking & Security ---
#   1 Application Load Balancer (ALB): Distributes traffic to the ECS tasks.
#   1 Target Group + Listener: Configures routing and health checks for the ALB.
#   3 Security Groups: Granular access control for the Builder, ALB, and Cluster
#   instances.
# --- Scaling & Observability ---
#   1 ScalableTarget + ScalingPolicy: Configures dynamic auto-scaling based on
#   traffic demand.
#   1 CloudWatch Dashboard: Provides real-time visibility into pipeline metrics and
#   cluster health.

# -----
# Parameters section
# -----
Parameters:
  NetworkStackName:
    Description: >-
```

Infraestructura para la Computación de Altas Prestaciones

```

    The logical name of the prerequisite network stack. This is required to
    import cross-stack values for the VPC and Subnet identifiers.
    Type: String
    MinLength: 1
    MaxLength: 255
    AllowedPattern: '^[a-zA-Z][-a-zA-Z0-9]*$'
    Default: proy-network

ECSAMI:
    Description: The SSM Parameter Store path to retrieve the latest Amazon Linux
    2023 AMI optimized for ECS workloads.
    Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
    Default: /aws/service/ecs/optimized-ami/amazon-linux-2023/recommended/image_id

# -----
# Outputs section
# -----
Outputs:
    ClusterName:
        Description: The ECS cluster into which to launch resources
        Value: ECSCluster

    CapacityProvider:
        Description: The cluster capacity provider that the service should use to
        request capacity when it wants to start up a task
        Value: CapacityProvider

    ECSTaskDefinition:
        Description: The created Taskdefinition.
        Value: !Ref TaskDefinition

# -----
# Resources section
# -----
Resources:

    # --- Security Groups Layer ---
    FlaskSecurityGroup:
        Type: AWS::EC2::SecurityGroup
        Properties:
            GroupDescription: Enable TCP ingress for Flask Builder
            VpcId:
                Fn::ImportValue: !Sub ${NetworkStackName}-VPCID
            Tags:

```


Infraestructura para la Computación de Altas Prestaciones

```
- Key: Name
  Value: proy-Flask Security Group
SecurityGroupIngress:
- IpProtocol: tcp
  FromPort: 5000
  ToPort: 5000
  CidrIp: 0.0.0.0/0

ALBSecurityGroup:
Type: AWS::EC2::SecurityGroup
Properties:
  GroupDescription: Public Access for Load Balancer
  VpcId:
    Fn::ImportValue: !Sub ${NetworkStackName}-VPCID
  Tags:
    - Key: Name
      Value: proy-ALB-Security-Group
  SecurityGroupIngress:
    - IpProtocol: tcp
      FromPort: 5000
      ToPort: 5000
      CidrIp: 0.0.0.0/0

PrivateFlaskSecurityGroup:
Type: AWS::EC2::SecurityGroup
DependsOn: ALBSecurityGroup
Properties:
  GroupDescription: Internal traffic from ALB to Cluster
  VpcId:
    Fn::ImportValue: !Sub ${NetworkStackName}-VPCID
  Tags:
    - Key: Name
      Value: proy-Private-Cluster-Security-Group
  SecurityGroupIngress:
    - IpProtocol: tcp
      FromPort: 5000
      ToPort: 5000
      SourceSecurityGroupId: !Ref ALBSecurityGroup

# --- Storage & Build Layer ---
Repository:
Type: AWS::ECR::Repository
Properties:
  RepositoryName: proy-repo
```

Infraestructura para la Computación de Altas Prestaciones

```
EmptyOnDelete: True
```

```
DockerInstance:
```

```
  Type: AWS::EC2::Instance
```

```
  Properties:
```

```
    InstanceType: t2.micro
```

```
    ImageId: ami-08a0d1e16fc3f61ea
```

```
    IamInstanceProfile: LabInstanceProfile
```

```
    Tags:
```

```
      - Key: Name
```

```
        Value: proy-DockerServer
```

```
  NetworkInterfaces:
```

```
    - GroupSet:
```

```
      - !Ref FlaskSecurityGroup
```

```
    AssociatePublicIpAddress: true
```

```
    DeviceIndex: 0
```

```
    DeleteOnTermination: true
```

```
    SubnetId:
```

```
      Fn::ImportValue: !Sub ${NetworkStackName}-PublicSubnet1ID
```

```
  UserData:
```

```
    Fn::Base64: !Sub |
```

```
      #!/bin/bash
```

```
      dnf update -y
```

```
      #install Docker
```

```
      dnf -y install docker
```

```
      #create a directory to store docker files
```

```
      mkdir -p /home/ec2-user/flask_docker
```

```
      #create 3 files in the previously created directory
```

```
      cat > /home/ec2-user/flask_docker/application.py<< EOF
```

```
      from flask import Flask, request, jsonify, redirect, url_for
      import boto3
```

```
      from boto3.dynamodb.conditions import Key
```

```
      from ec2_metadata import ec2_metadata
```

```
      import os
```

```
      TABLE = 'proy-Resultados'
```

```
      db = boto3.resource('dynamodb', region_name='us-east-1')
```

```
      table = db.Table(TABLE)
```

```
      ipv4 = ec2_metadata.private_ipv4
```

Infraestructura para la Computación de Altas Prestaciones

```
app = Flask(__name__)

@app.route('/')
def index():
    title = '<h1> No sabes hacer consultas pequeñin </h1>'
    content = '<h2> Dato curioso: LeBron James ha jugado contra más del
35% de todos los jugadores que han pisado una cancha en la historia de la NBA.
</h2>'

    return title + content

@app.route('/maxdiff', methods=['GET'])
def maxdiff():
    month = request.args['month']
    month = f'{int(month):02}'
    year = request.args['year']
    date = '-'.join([year, month])
    item =
table.query(KeyConditionExpression=Key('YearMonth').eq(date))['Items'][0]

    result = {}
    if item:
        maxdiff = item['maxdiff']
        result['Date'] = date
        result['Maxdiff'] = maxdiff
        result['ip'] = ipv4
        result = jsonify(result)
    else:
        result = '<h1>We currently do not have data of the requested date
:(...</h1>'

    return result

@app.route('/sd')
def sd():
    month = request.args['month']
    month = f'{int(month):02}'
    year = request.args['year']
    date = '-'.join([year, month])
```

Infraestructura para la Computación de Altas Prestaciones

```
        item =
table.query(KeyConditionExpression=Key('YearMonth').eq(date))['Items'][0]

        result = {}
        if item:
            sd = item['sd']
            result['Date'] = date
            result['sd'] = sd
            result['ip'] = ipv4
            result = jsonify(result)
        else:
            result = '<h1>We currently do not have data of the requested date
:(<...</h1>'

        return result

@app.route('/temp')
def temp():
    month = request.args['month']
    month = f'{int(month):02}'
    year = request.args['year']
    date = '-'.join([year, month])
    item =
table.query(KeyConditionExpression=Key('YearMonth').eq(date))['Items'][0]

    result = {}
    if item:
        temp = item['temp']
        result['Date'] = date
        result['temp'] = temp
        result['ip'] = ipv4
        result = jsonify(result)
    else:
        result = '<h1>We currently do not have data of the requested date
:(<...</h1>'

    return result

@app.errorhandler(Exception)
def handle_unexpected_error(error):
    return redirect(url_for('index'))
```

Infraestructura para la Computación de Altas Prestaciones

```
if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(debug=True, host='0.0.0.0', port=port)
EOF

cat > /home/ec2-user/flask_docker/requirements.txt<< EOF
Flask==3.1.0
Werkzeug==3.1.3
Jinja2==3.1.4
MarkupSafe==2.1.5
itsdangerous==2.2.0
click==8.1.7
boto3==1.35.71
botocore==1.35.71
jmespath==1.0.1
s3transfer==0.10.4
ec2_metadata==2.14.0
requests==2.32.3
EOF

cat > /home/ec2-user/flask_docker/Dockerfile<< EOF
# syntax=docker/dockerfile:1.4
FROM python:3.9-alpine
WORKDIR /app
COPY requirements.txt requirements.txt
RUN pip3 install -r requirements.txt
COPY . .
CMD [ "python3", "application.py" ]
EOF

#change cwd to the previously created directory
cd /home/ec2-user/flask_docker

#start docker
sudo systemctl start docker

#build the container
sudo docker build -t flask_container .

ACCID=$(aws sts get-caller-identity --query Account --output text)

sudo aws ecr get-login-password --region us-east-1 | sudo docker login --
username AWS --password-stdin "$ACCID".dkr.ecr.us-east-1.amazonaws.com
```

Infraestructura para la Computación de Altas Prestaciones

```
sudo docker tag flask_container "$ACCID".dkr.ecr.us-east-1.amazonaws.com/proy-repo:flask_container
sudo docker push "$ACCID".dkr.ecr.us-east-1.amazonaws.com/proy-repo:flask_container

# --- Compute & Orchestration Layer ---
ECSCluster:
  Type: AWS::ECS::Cluster
  Properties:
    ClusterName: proy-ecs-cluster
    ClusterSettings:
      - Name: containerInsights
        Value: disabled

ContainerInstances:
  Type: AWS::EC2::LaunchTemplate
  Properties:
    LaunchTemplateName: "asg-launch-template"
    LaunchTemplateData:
      ImageId:
        Ref: ECSAMI
      InstanceType: t2.micro
      IamInstanceProfile:
        Arn: !Sub arn:aws:iam::${AWS::AccountId}:instance-profile/LabInstanceProfile
      SecurityGroupIds:
        - !Ref PrivateFlaskSecurityGroup
      UserData:
        Fn::Base64: !Sub |
          #!/bin/bash -xe
          echo ECS_CLUSTER=${ECSCluster} >> /etc/ecs/ecs.config
          yum install -y aws-cfn-bootstrap
          /opt/aws/bin/cfn-init -v --stack ${AWS::StackId} --resource
ContainerInstances --configsets full_install --region ${AWS::Region} &
      MetadataOptions:
        HttpEndpoint: enabled
        HttpTokens: required

ECSAutoScalingGroup:
  Type: AWS::AutoScaling::AutoScalingGroup
  DependsOn:
    - ECSCluster
  Properties:
    VPCZoneIdentifier:
```

Infraestructura para la Computación de Altas Prestaciones

```

- Fn::ImportValue: !Sub ${NetworkStackName}-PrivateSubnet1ID
- Fn::ImportValue: !Sub ${NetworkStackName}-PrivateSubnet2ID
LaunchTemplate:
  LaunchTemplateId: !Ref ContainerInstances
  Version: !GetAtt ContainerInstances.LatestVersionNumber
  MinSize: 4
  MaxSize: 6
  NewInstancesProtectedFromScaleIn: true
UpdatePolicy:
  AutoScalingReplacingUpdate:
    WillReplace: "true"

CapacityProvider:
  Type: AWS::ECS::CapacityProvider
  Properties:
    AutoScalingGroupProvider:
      AutoScalingGroupArn: !Ref ECSAutoScalingGroup
      ManagedScaling:
        InstanceWarmupPeriod: 60
        MinimumScalingStepSize: 1
        MaximumScalingStepSize: 4
        Status: ENABLED
        TargetCapacity: 100
      ManagedTerminationProtection: ENABLED

CapacityProviderAssociation:
  Type: AWS::ECS::ClusterCapacityProviderAssociations
  Properties:
    CapacityProviders:
      - !Ref CapacityProvider
    Cluster: !Ref ECSCluster
    DefaultCapacityProviderStrategy:
      - Base: 0
        CapacityProvider: !Ref CapacityProvider
        Weight: 1

# --- Application Layer ---
TaskDefinition:
  Type: 'AWS::ECS::TaskDefinition'
  Properties:
    Family: proy-task
    RequiresCompatibilities:
      - EC2
    NetworkMode: awsvpc

```

Infraestructura para la Computación de Altas Prestaciones

```
Cpu: .25 vCPU
Memory: 0.5 GB
TaskRoleArn: LabRole
ExecutionRoleArn: LabRole
ContainerDefinitions:
  - Name: proy-container
    Image: !Sub ${AWS::AccountId}.dkr.ecr.us-east-1.amazonaws.com/proy-
repo:flask_container
    Cpu: 256
    PortMappings:
      - ContainerPort: 5000
        Protocol: tcp
        Name: flasktraffic
        AppProtocol: http

ALB:
  Type: AWS::ElasticLoadBalancingV2::LoadBalancer
  Properties:
    Name: flask-load-balancer
    IpAddressType: ipv4
    SecurityGroups:
      - !Ref ALBSecurityGroup
    Subnets:
      - Fn::ImportValue: !Sub ${NetworkStackName}-PublicSubnet1ID
      - Fn::ImportValue: !Sub ${NetworkStackName}-PublicSubnet2ID

IPTargetGroup:
  Type: AWS::ElasticLoadBalancingV2::TargetGroup
  Properties:
    TargetType: ip
    Name: ip-target-group
    Protocol: HTTP
    Port: 5000
    VpcId:
      Fn::ImportValue:
        !Sub ${NetworkStackName}-VPCID
    HealthyThresholdCount: 2
    HealthCheckIntervalSeconds: 6

HTTPListener:
  Type: "AWS::ElasticLoadBalancingV2::Listener"
  Properties:
    DefaultActions:
      - Type: "forward"
```


Infraestructura para la Computación de Altas Prestaciones

```
    TargetGroupArn: !Ref IPTargetGroup
    LoadBalancerArn: !Ref ALB
    Port: 5000
    Protocol: "HTTP"

ECSService:
  Type: AWS::ECS::Service
  DependsOn:
    - HTTPListener
  Properties:
    Cluster: !Ref ECSCluster
    ServiceName: proy-ecs-service
    SchedulingStrategy: REPLICA
    DesiredCount: 4
    AvailabilityZoneRebalancing: ENABLED
    LoadBalancers:
      - ContainerName: proy-container
        ContainerPort: 5000
        TargetGroupArn: !Ref IPTargetGroup
    NetworkConfiguration:
      AwsvpcConfiguration:
        SecurityGroups:
          - !Ref PrivateFlaskSecurityGroup
        Subnets:
          - Fn::ImportValue: !Sub ${NetworkStackName}-PrivateSubnet1ID
          - Fn::ImportValue: !Sub ${NetworkStackName}-PrivateSubnet2ID
    TaskDefinition: !Ref TaskDefinition

# --- Scaling Policy & Ops ---
ECSScalableTarget:
  Type: AWS::ApplicationAutoScaling::ScalableTarget
  DependsOn:
    - ECSService
  Properties:
    MaxCapacity: 6
    MinCapacity: 4
    ServiceNamespace: ecs
    ScalableDimension: 'ecs:service:DesiredCount'
    ResourceId: 'service/proy-ecs-cluster/proy-ecs-service'

ServiceScalingPolicyALB:
  Type: AWS::ApplicationAutoScaling::ScalingPolicy
  Properties:
    PolicyName: ALB-ECS-Policy
```

Infraestructura para la Computación de Altas Prestaciones

```
PolicyType: TargetTrackingScaling
ScalingTargetId: !Ref ECSScalableTarget
TargetTrackingScalingPolicyConfiguration:
  TargetValue: 1000
  ScaleInCooldown: 180
  ScaleOutCooldown: 30
  DisableScaleIn: true
  PredefinedMetricSpecification:
    PredefinedMetricType: ALBRequestCountPerTarget
    ResourceLabel: !Join
      - '/'
      - - !GetAtt ALB.LoadBalancerFullName
        - !GetAtt IPTargetGroup.TargetGroupName

# Observability Dashboard
ProjectDashboard:
  Type: AWS::CloudWatch::Dashboard
  Properties:
    DashboardName: AquaSense-Operational-Dashboard
    DashboardBody: !Sub |
      {
        "widgets": [
          {
            "type": "metric",
            "x": 0,
            "y": 0,
            "width": 12,
            "height": 6,
            "properties": {
              "metrics": [
                [ "AWS/Lambda", "Invocations", "FunctionName", "lambda1", {
"stat": "Sum", "period": 300, "label": "Ingesta (Lambda1)" } ],
                [ "...", "lambda2", { "stat": "Sum", "period": 300, "label":
"Procesamiento (Lambda2)" } ],
                [ "...", "lambda3", { "stat": "Sum", "period": 300, "label":
"Alertas (Lambda3)" } ]
              ],
              "view": "timeSeries",
              "stacked": false,
              "region": "${AWS::Region}",
              "title": "Actividad del Pipeline de Datos (Invocaciones)"
            }
          ],
          {

```

Infraestructura para la Computación de Altas Prestaciones

```
    "type": "metric",
    "x": 12,
    "y": 0,
    "width": 12,
    "height": 6,
    "properties": {
      "metrics": [
        [ "AWS/ECS", "CPUUtilization", "ClusterName", "${ECSCluster}", {
"stat": "Average", "period": 300 } ],
        [ ".", "MemoryUtilization", ".", ".", { "stat": "Average",
"period": 300 } ]
      ],
      "view": "timeSeries",
      "stacked": false,
      "region": "${AWS::Region}",
      "title": "Salud del Clúster ECS"
    }
  }
]
```

Proy-icap-pipeline

```
AWSTemplateFormatVersion: 2010-09-09
Description: >-
  Pipeline Template: Serverless Data Pipeline definition (S3, Lambda, DynamoDB,
  SNS).

# -----
# Outputs section
# -----
Outputs:
  Message:
    Description: Manual Trigger Instruction
    Value: !Sub El bucket S3 '${S3Bucket}' debe establecerse MANUALMENTE como
    trigger de la función '${Lambda1Function}' debido a limitaciones de permisos.

# -----
# Resources section
# -----
Resources:
```

Infraestructura para la Computación de Altas Prestaciones

```
# --- Notifications Layer ---
SNSTopic:
  Type: AWS::SNS::Topic
  Properties:
    TopicName: DeviationExceeded

EmailSubscription:
  Type: AWS::SNS::Subscription
  Properties:
    TopicArn: !Ref SNSTopic
    Protocol: email
    Endpoint: jlabellan@um.es

# --- Storage Layer (S3 & NoSQL) ---
S3Bucket:
  Type: AWS::S3::Bucket
  Properties:
    BucketName: proydatabucket-javier-alvaro-v2

DynamoDBTable1:
  Type: AWS::DynamoDB::Table
  Properties:
    TableName: proy-Datos
    AttributeDefinitions:
      - AttributeName: YearMonth
        AttributeType: S
      - AttributeName: Day
        AttributeType: S
    KeySchema:
      - AttributeName: YearMonth
        KeyType: HASH
      - AttributeName: Day
        KeyType: RANGE
    ProvisionedThroughput:
      ReadCapacityUnits: 10
      WriteCapacityUnits: 10
    StreamSpecification:
      StreamViewType: NEW_IMAGE

DynamoDBTable2:
  Type: AWS::DynamoDB::Table
  Properties:
    TableName: proy-Resultados
    AttributeDefinitions:
```

Infraestructura para la Computación de Altas Prestaciones

```
- AttributeName: YearMonth
  AttributeType: S
KeySchema:
  - AttributeName: YearMonth
    KeyType: HASH
ProvisionedThroughput:
  ReadCapacityUnits: 10
  WriteCapacityUnits: 10

# --- Compute Layer (Lambdas) ---
Lambda1Function:
  Type: AWS::Lambda::Function
  Properties:
    FunctionName: lambda1
    Handler: index.lambda_handler
    Role: !Sub arn:aws:iam::${AWS::AccountId}:role/LabRole
    Runtime: python3.12
    Timeout: 20
    Code:
      ZipFile: |
        import json, urllib, boto3, csv
        import datetime
        from decimal import Decimal
        TABLE_NAME = 'proy-Datos'
        BUCKET_NAME = 'proydatabucket-javier-alvaro-v2'
        s3 = boto3.resource('s3')
        dynamodb = boto3.resource('dynamodb', region_name='us-east-1')
        table = dynamodb.Table(TABLE_NAME)
        def lambda_handler(event, context):
            bucket = BUCKET_NAME
            key =
urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])
            localFilename = '/tmp/Temperaturas.csv'
            try:
                s3.meta.client.download_file(bucket, key, localFilename)
            except Exception as e:
                print('Error getting object {} from bucket {}. Make sure they exist
and your bucket is in the same region as this function.'.format(key, bucket))
                raise e
            with open(localFilename) as csvfile:
                reader = csv.DictReader(csvfile, delimiter=',')
                items = {}
                for row in reader:
                    try:
```

Infraestructura para la Computación de Altas Prestaciones

```
        fecha =  
str(datetime.datetime.strptime(row['Fecha'], '%Y/%m/%d').date()).split('-')  
        year_month = '-'.join(fecha[:2])  
        day = fecha[2]  
        items[(year_month, day)] = {  
            'YearMonth': year_month,  
            'Day': day,  
            'Medias': float(row['Medias']),  
            'Desviaciones': float(row['Desviaciones'])  
        }  
    except Exception as e:  
        print(e)  
        print("Unable to insert data into DynamoDB table".format(e))  
    with table.batch_writer() as batch:  
        for _, item in items.items():  
            batch.put_item(Item=json.loads(json.dumps(item),  
parse_float=Decimal))  
    return 'FINISHED!'
```

Lambda2Function:

Type: AWS::Lambda::Function

Properties:

FunctionName: lambda2

Handler: index.lambda_handler

Role: !Sub arn:aws:iam::\${AWS::AccountId}:role/LabRole

Runtime: python3.12

Timeout: 20

Code:

ZipFile: |

```
import boto3  
from boto3.dynamodb.conditions import Key, Attr  
from decimal import Decimal  
import json
```

TABLE1_NAME = 'proy-Datos'

TABLE2_NAME = 'proy-Resultados'

dynamodb = boto3.resource('dynamodb')

table1 = dynamodb.Table(TABLE1_NAME)

table2 = dynamodb.Table(TABLE2_NAME)

def get_previous_month(date):

previous_month = date.split('-')

Infraestructura para la Computación de Altas Prestaciones

```
    if previous_month[1] == '01':
        previous_month[0] = str(int(previous_month[0])-1)
        previous_month[1] = '12'

    else:
        previous_month[1] = f'{(int(previous_month[1])-1):02}'

    return '-'.join(previous_month)

def get_next_month(date):
    next_month = date.split('-')
    if next_month[1] == '12':
        next_month[0] = str(int(next_month[0])+1)
        next_month[1] = '01'

    else:
        next_month[1] = f'{(int(next_month[1])+1):02}'

    return '-'.join(next_month)

def get_items(current_date):
    items =
table1.query(KeyConditionExpression=Key('YearMonth').eq(current_date))['Items']
    return items

def get_metrics(items, previous_month_items):
    previous_month_avg = 0.0
    if previous_month_items:
        for item in previous_month_items:
            if float(item['Medias']) > previous_month_avg:
                previous_month_avg = float(item['Medias'])

    medias = [float(items[i]['Medias']) for i in range(len(items))]
    desviaciones = [float(items[i]['Desviaciones']) for i in
range(len(items))]

    max_sd = max(desviaciones)
    media_mensual = sum(medias) / len(medias)

    if previous_month_avg > 0:
        diff = max(medias) - previous_month_avg
    else:
```

Infraestructura para la Computación de Altas Prestaciones

```
        diff = 0.0

    return {'YearMonth' : items[0]['YearMonth'],
            'maxdiff' : float(diff),
            'sd' : float(max_sd),
            'temp' :float(media_mensual)}

def lambda_handler(event, context):

    inserted_or_modified = set()

    for e in event['Records']:
        date = e['dynamodb']['Keys']['YearMonth']['S']
        inserted_or_modified.add(date)

    inserted_or_modified = list(inserted_or_modified)
    new_rows = []

    for date in inserted_or_modified:
        items = get_items(date)
        previous_month_items = get_items(get_previous_month(date))
        row = get_metrics(items=items,
previous_month_items=previous_month_items)
        new_rows.append(row)

        next_month_items = get_items(get_next_month(date))
        if next_month_items:
            next_month_row = get_metrics(items=next_month_items,
previous_month_items=items)
            new_rows.append(next_month_row)

    for row in new_rows:
        table2.put_item(Item= json.loads(json.dumps(row),
parse_float=Decimal))

    return 'FINISHED!'
```

Lambda3Function:

Type: AWS::Lambda::Function

Properties:

FunctionName: lambda3

Handler: index.lambda_handler

Role: !Sub arn:aws:iam::\${AWS::AccountId}:role/LabRole

Runtime: python3.12

Infraestructura para la Computación de Altas Prestaciones

```
Timeout: 20
Code:
ZipFile: |
    import boto3
    import json
    def lambda_handler(event, context):
        message = None
        fechas = []
        desviaciones = []
        for record in event['Records']:
            newImage = record['dynamodb'].get('NewImage', None)
            if newImage:
                desviacion =
float(record['dynamodb']['NewImage']['Desviaciones']['N'])
                if desviacion is not None and float(desviacion) > 0.5:
                    fecha = record['dynamodb']['NewImage']['YearMonth']['S']
                    dia = record['dynamodb']['NewImage']['Day']['S']
                    fechas.append(f"{fecha}-{dia}")
                    desviaciones.append(desviacion)
        if fechas and desviaciones:
            sns = boto3.client('sns')
            alertTopic = 'DeviationExceeded'
            snsTopicArn = [t['TopicArn'] for t in sns.list_topics()['Topics']
if t['TopicArn'].lower().endswith(':'+alertTopic.lower())][0]
            message = "Deviation exceeded on the following dates:\n" +
"\n".join([f"{fecha}: {desviacion:.3f}" for fecha, desviacion in zip(fechas,
desviaciones)])
            sns.publish(
                TopicArn=snsTopicArn,
                Subject='Desviación Alta Detectada',
                Message=message,
                MessageStructure='raw'
            )
        return 'Updated'

# --- Event Triggers ---
Lambda2Trigger:
    Type: AWS::Lambda::EventSourceMapping
    Properties:
        EventSourceArn: !GetAtt DynamoDBTable1.StreamArn
        FunctionName: !Ref Lambda2Function
        Enabled: 'True'
        StartingPosition: TRIM_HORIZON
        BatchSize: 100
```

Infraestructura para la Computación de Altas Prestaciones

```
Lambda3Trigger:
  Type: AWS::Lambda::EventSourceMapping
  Properties:
    EventSourceArn: !GetAtt DynamoDBTable1.StreamArn
    FunctionName: !Ref Lambda3Function
    Enabled: 'True'
    StartingPosition: TRIM_HORIZON
    BatchSize: 100
```